# Informatique quantique 2

# Classical heuristics for optimization

Valentin Gilbert – Quantum computing consultant at Qounselor

2025-2026

# Plan of session 2

1- Introduction to optimization methods:

- Deterministic and Stochastic methods
- Examples

2- Gradient descent

3- Simulated annealing

4- Branch and bound method

# Introduction

# Optimization Methods

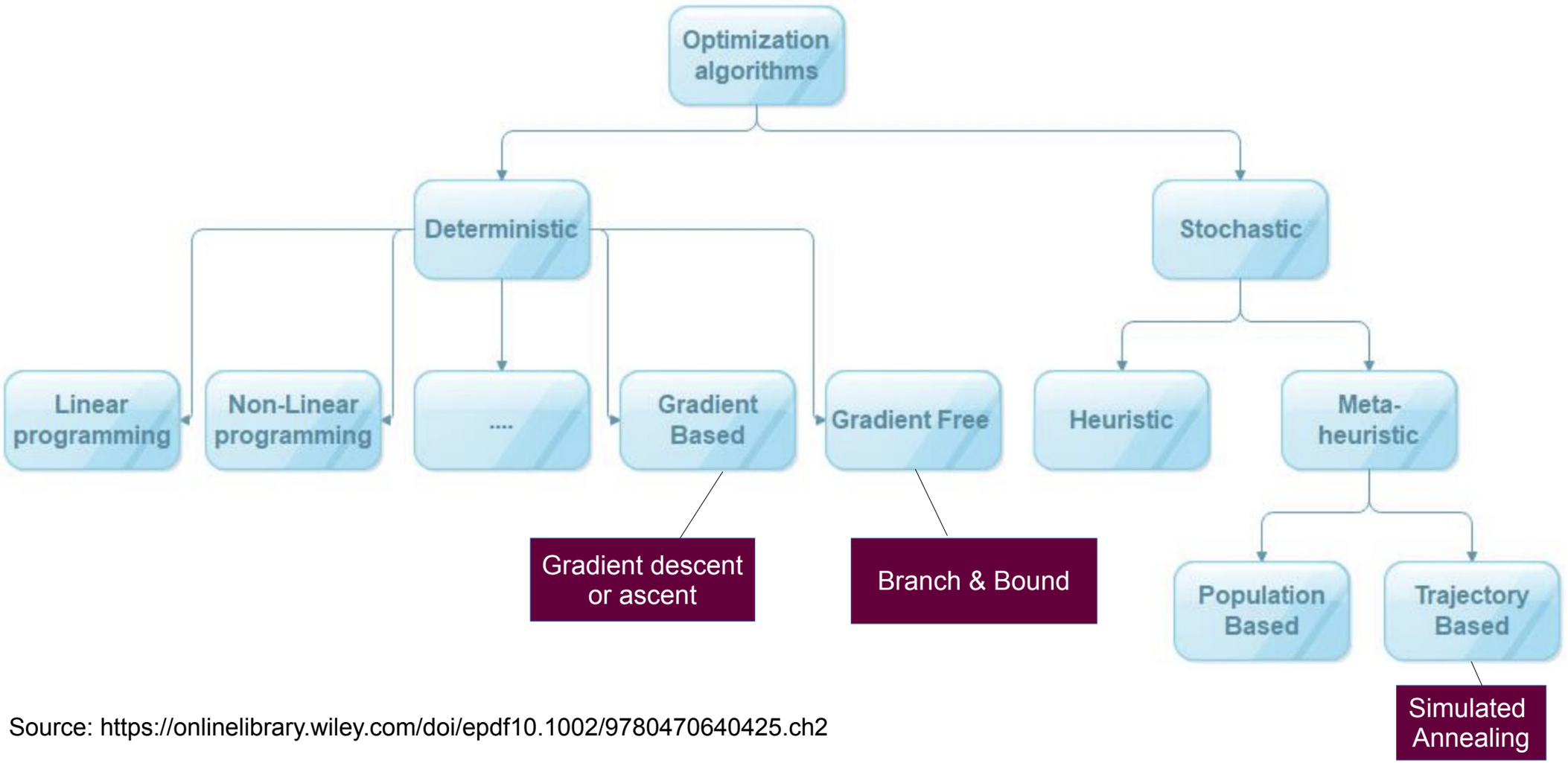
Types of optimization methods:

- **Deterministic methods:**
  - Refers to mathematical programming (linear programming, simplex method ...)
  - Provides a theoretical guarantee concerning the solution that has been found
  - Behavior fully predictable with a precise sequence of steps for the exploration of the search space.

- **Stochastic methods:**
  - Incorporates randomness in the optimization process. The randomness can be located at different steps :
    - Randomization concerning the evaluation of the objective function (presence of noise => The same input may not produce the same output)
    - Randomization during the iterations :
      - Random selection of neighbor solutions
      - Acceptance criteria of a solution based on a random acceptance law

# Optimization Methods

Classification example :




Source: https://onlinelibrary.wiley.com/doi/epdf10.1002/9780470640425.ch2

# Heuristics

Definition: A heuristic is a problem-tailored method that will hopefully give a good result.

Why do we use heuristics :

- Complex problems (large, non-linear, non-convex problems with lots of local minima).
- To solve multivariate combinatorial optimization problems that are difficult to solve classically.

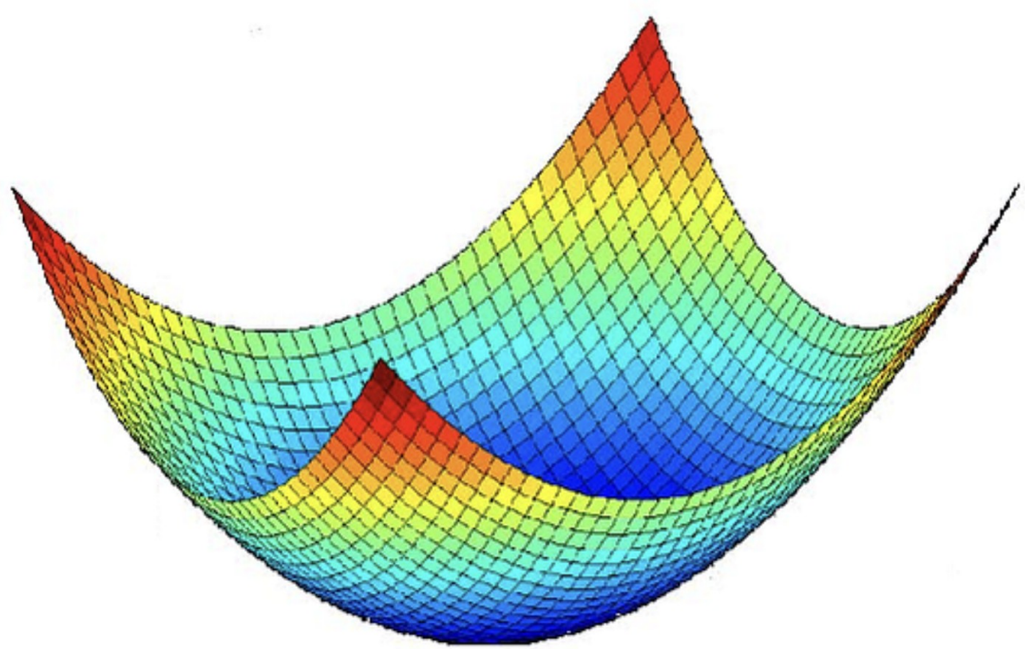
Some properties:

- Unlike deterministic methods, heuristics do  guarantee the quality of the result.
- Dilemma between mathematicians (proofs) and engineers (something that works in practice).
- Heuristics are good for dealing with local optima without getting stuck in them.

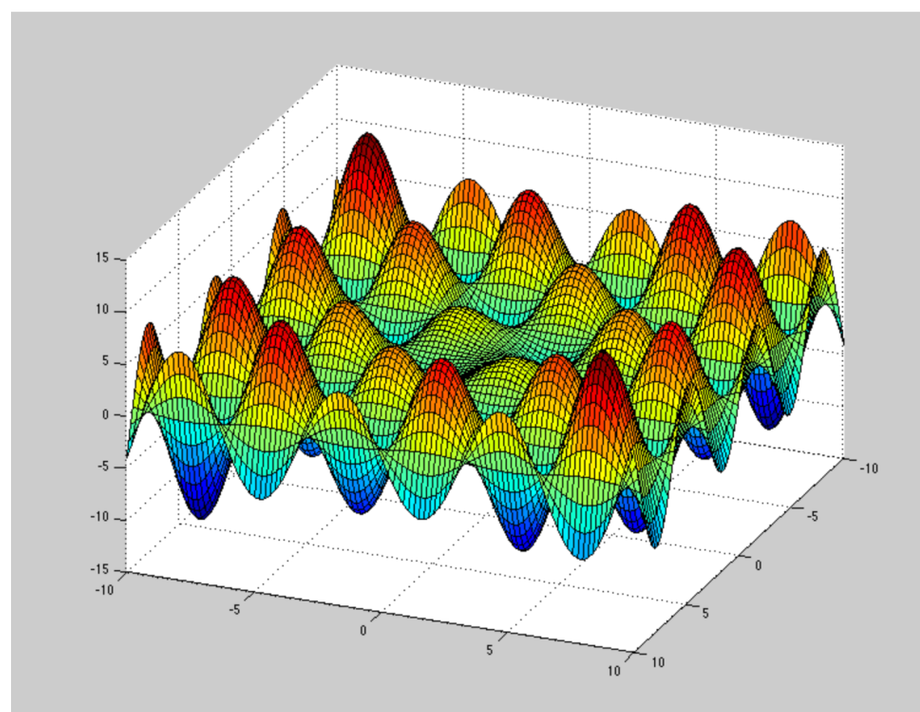
Metaheuristics: Algorithms that are more general than heuristics. These methods are expected to work well for a wide range of problems (e.g., Simulated Annealing)

# Heuristics

<u>Convex vs non-convex :</u>
Convex functions are shaped like a cup ⋃ while concave functions are shaped like a cap ⋂



Convex function




Highly non convex function

# Heuristics

Types of heuristic methods:

- **Construction method** : Starts with an empty solution at the beginning and iteratively extends the solution until a complete solution is obtained.
- **Improvement method** : Find a feasible and complete solution that is then slightly altered (with local moves) to improve the quality of the solution.

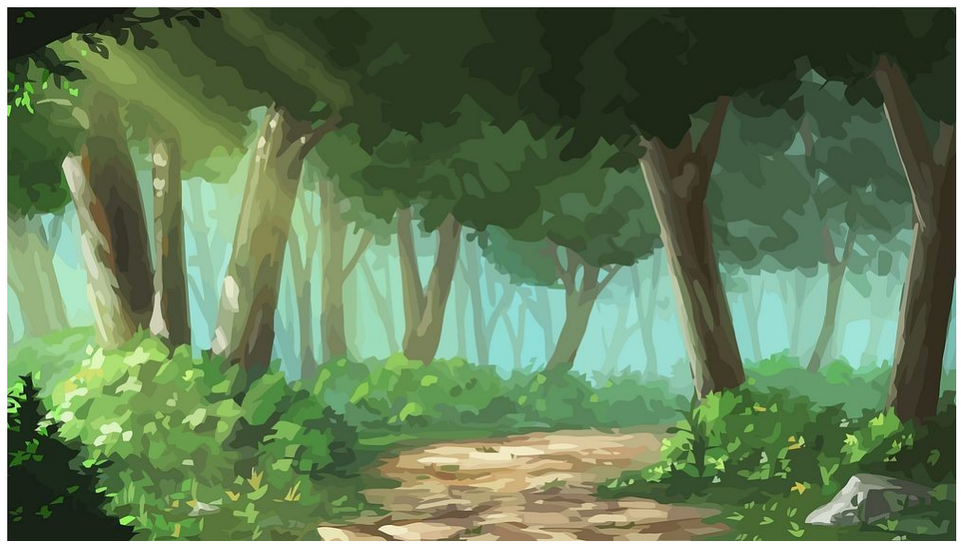
Example of famous heuristics:

- Simulated annealing (1983) - https://www.science.org/doi/10.1126/science.220.4598.671
- Tabu search (1986) - https://pubsonline.informs.org/doi/epdf/10.1287/ijoc.1.3.190
- Genetic algorithm
- Other more recent methods : Particle swarm optimization methods.

Albeit old, these methods are still used a lot in practice (easy to code, easy to use)

# Gradient descent

# Gradient descent



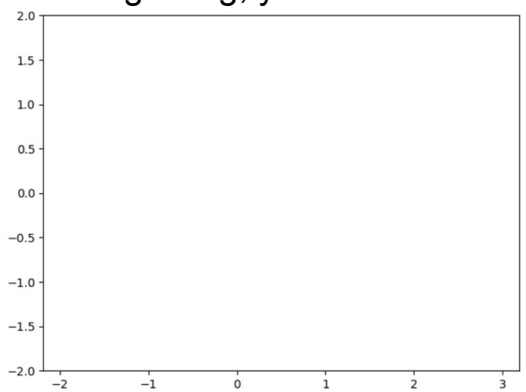
Source of the images:
https://freesvg.org/jungle-explorer
https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTGVYcVJS3xwm2SE-cl3zRQI6EKvh-VUc2VsQ&s
https://www.rawpixel.com/image/10165996/image-plant-grass-leaf

# Gradient descent

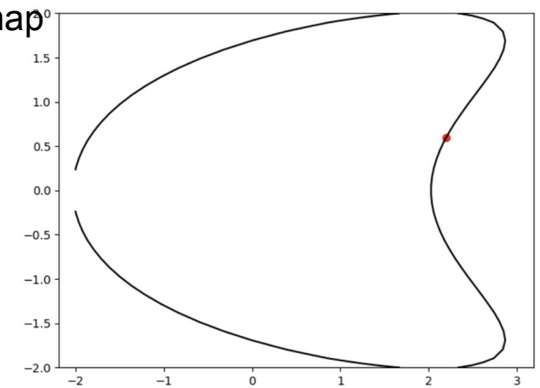
<u>Definition:</u> Given a function $f$ on two variables $(a, b)$: try to find the point of highest value

At coordinates $(a_{\max}, b_{\max})$

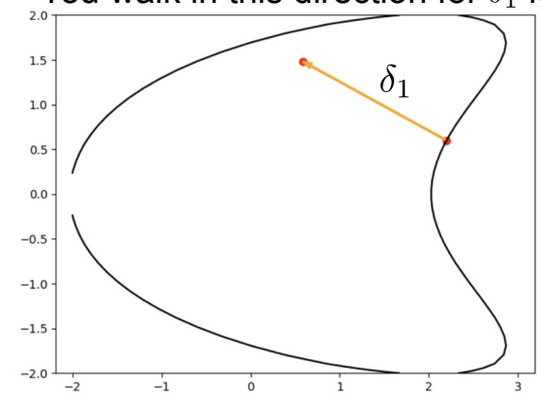
At the beginning, you start from nothing



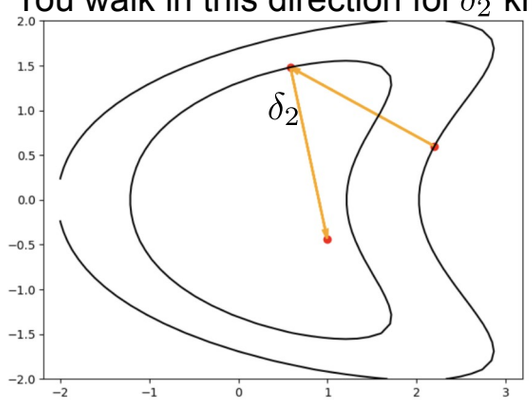
You start from a random point on the map



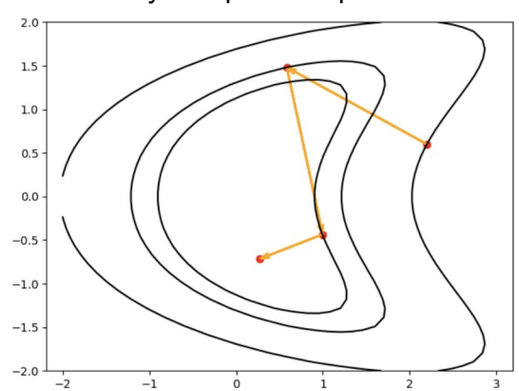
You then look around you to choose the steepest climb direction.
You walk in this direction for $\delta_1$ km



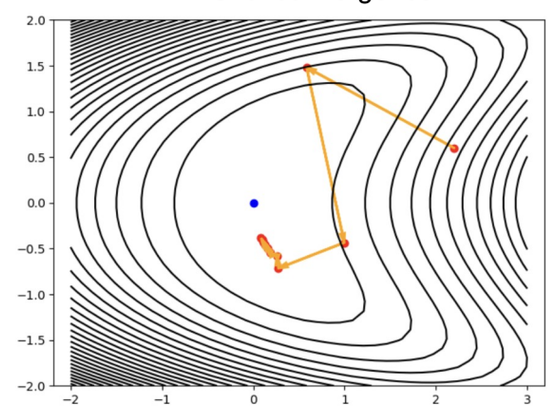

You then look again around you to choose the steepest climb direction.
You walk in this direction for $\delta_2$ km




And you repeat this process ...



Until convergence



# Gradient descent

Motivation:
The aim is to solve a problem with continuous variables. We consider we have access to the function and that the function can be derived

Main strength of Gradient descent method:
- Fast convergence to local or global optima
- Parameters of the gradient descent can be optimized to avoid local optima
- Scale easily to a huge number of variables

Method that have shown its value in practice:
- Employed in many industrial use cases
- Employed to optimize neural network (reduction of error for the prediction of a result)

# Gradient descent

Example with two variables:

$$f(a,b) = a^2 + 3b^2$$

Aim: Find the smallest minimum value of this function over a specific interval with a gradient descent

Small recall about derivatives:

For this function the gradient is given by:

$$\mathrm{grad} f(a,b) = \nabla f(a,b) = \left(\frac{\partial f}{\partial a}(a,b), \frac{\partial f}{\partial b}(a,b)\right)$$

nabla

partial derivatives

For a single variable function, the derivative gives the Slope of the curve (related to the steepness)

Gradient: Simply the generalization of derivatives to n-variable functions (n-degree of freedom)

$$\nabla f(a,b) = (2a, 6b)$$

# Gradient descent

<u>Example with two variables:</u> $f(a,b) = a^2 + 3b^2$

- We start at a random point: let's say $(a_0, b_0) = (2, 1)$ $\quad f(a_0, b_0) = 2^2 + 3 \times 1^1 = 7$

- We compute the gradient at this point $\nabla f(a_0, b_0) = (4, 6)$

- We fix a parameter $\delta_0 = 0.2$ (distance travelled)

- We compute the coordinates of the new point (we take the opposite direction of the gradient as it's a minimization problem):

$$(a_1, b_1) = (2, 1) - 0.2 \times (4, 6) = (1.2, -0.2)$$

$$f(a_1, b_1) = 1.2^2 + 3 \times (-0.2)^2 = 1.56$$

=> We decreased the value of $f$

**Exercise:** Repeat this step 3 times

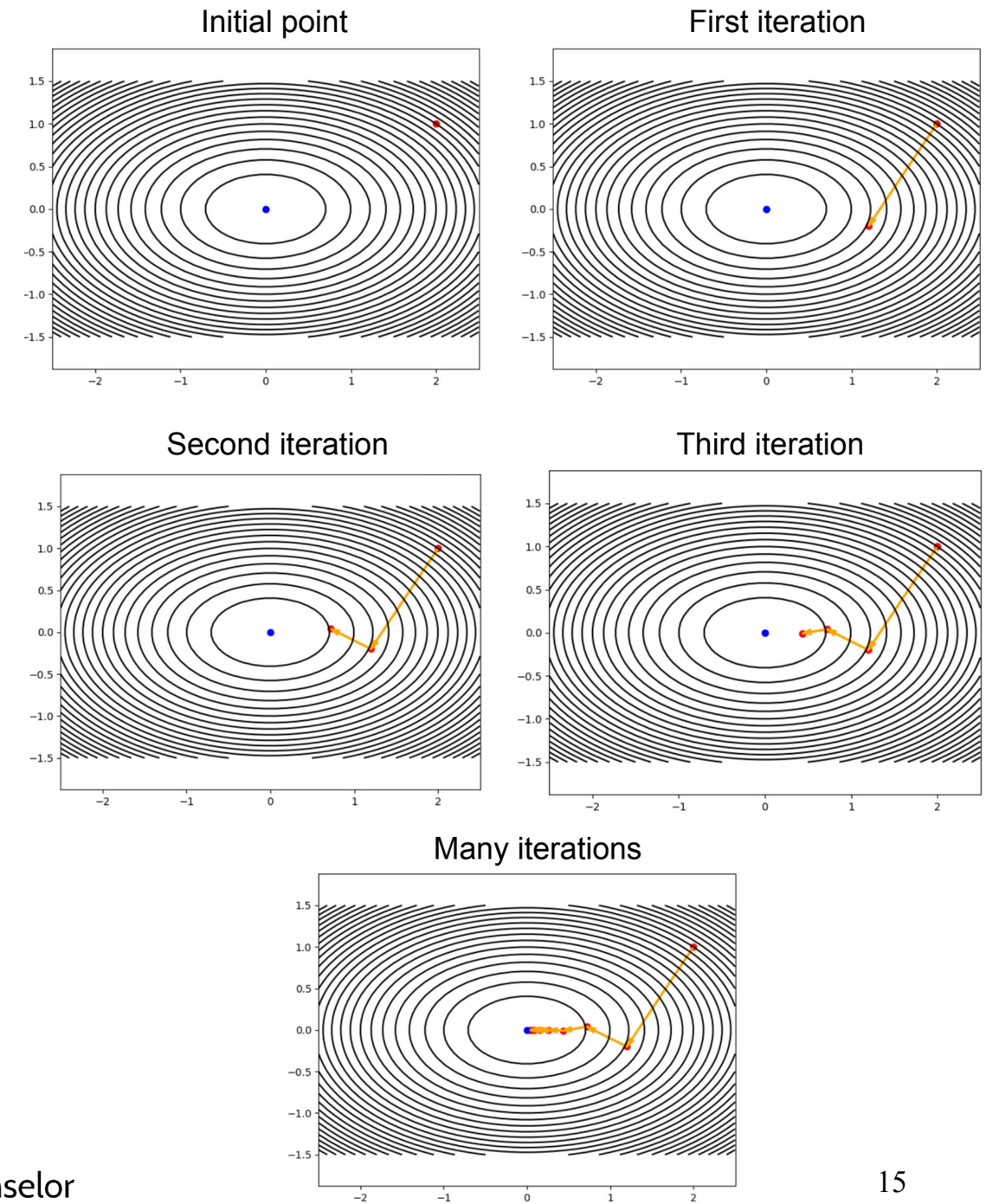
# Gradient descent

<u>Example with two variables:</u>

$$f(a,b) = a^2 + 3b^2$$

| $k$ | $(a_k, b_k)$ | $\operatorname{grad} f(a_k, b_k)$ | $f(a_k, b_k)$ |
|---|---|---|---|
| 0 | $(2, 1)$ | $(4, 6)$ | 7 |
| 1 | $(1.2, -0.2)$ | $(2.4, -1.20)$ | 1.56 |
| 2 | $(0.72, 0.04)$ | $(1.44, 0.24)$ | 0.523 |
| 3 | $(0.432, -0.008)$ | $(0.864, -0.048)$ | 0.186 |
| 4 | $(0.2592, 0.0016)$ | $(0.5184, 0.0096)$ | 0.067 |
| 5 | $(0.15552, -0.00032)$ | $(0.31104, -0.00192)$ | 0.024 |
| ... | | | |
| 10 | $(0.012, 1.02 \cdot 10^{-7})$ | $(0.024, 6.14 \cdot 10^{-7})$ | 0.00014 |
| ... | | | |
| 20 | $(7.31 \cdot 10^{-5}, 1.04 \cdot 10^{-14})$ | $(1.46 \cdot 10^{-4}, 6.29 \cdot 10^{-14})$ | $5.34 \cdot 10^{-9}$ |

=> The minimum of the function is reached at $(a,b) = (0,0)$



# Gradient descent

<u>Gradient descent algorithm:</u>

- Recurrent function:

$$(a_{k+1}, b_{k+1}) = (a_k, b_k) - \delta \nabla f(a_k, b_k)$$

- $\delta$ is the step, usually chosen to be small enough.

- The gradient of $f$ diminishes when we are reaching the minimum of the function (space between two points gets smaller and smaller).

<u>Issues met by the gradient descent method:</u>
- The gradient descent method is a **local** heuristic. It is possible to be trapped in local minima.

- Saddle points (neither minimum nor maximum) where the derivative is equal to 0. Big issue seen in parameterized quantum algorithms (see lecture on the QAOA).

# Gradient descent

<u>Gradient descent algorithm:</u>

- Recurrent function:

$$(a_{k+1}, b_{k+1}) = (a_k, b_k) - \delta \nabla f(a_k, b_k)$$

- $\delta$ is the step, usually chosen to be small enough.

- The gradient of $f$ diminishes when we are reaching the minimum of the function (space between two points gets smaller and smaller).

<u>How to choose $\delta$ (called the *learning rate*)?</u>

- Even if $\delta$ is constant, $\nabla f$ tend to be 0 when we are near to a minimum.
- But, we cannot choose $\delta$ too big (we would oscillate around the minimum) and not too small (otherwise, the heuristic would be very slow).

# Gradient descent

<u>How to choose $\delta$ ?</u>

- Linear decrease: $\delta_k = \dfrac{\delta_0}{k+1}$

- Quadratic decrease: $\delta_k = \dfrac{\delta_0}{(k+1)^2}$

- Exponential decrease: $\delta_k = \delta_0 \beta^k$
  where $\beta$ is a small positive constant <1

<u>When is done the decrease ?</u>
- After every iteration
- After a fixed number of steps (step decay)
- Per epoch (specific to machine learning, defines a complete pass through the entire dataset)

Finding the ideal optimization parameters is hard in general

# Gradient descent

<u>When does the heuristic stops ?</u>

- When the value of the gradient is very small (remember, we could be located in a saddle point and not in a local minima).
- In practice, we could define a maximum number of iterations.

**Exercise:** Write the pseudo code of this algorithm. The pseudo code should include:

- The intialization
- The gradient algorithm
- A decrease law of $\delta$ which decrease after a specific number of steps
- Some stopping condition

# Gradient descent

<u>Gradient descent algorithm pseudocode:</u>

- n = CONSTANT_NUMBER_STEP
- current_conf = set_random()  // Vector of values
- learning_law = quadratic_learning(starting_point) // Initialize the learning law
- While not termination_criteria:
  - For i in (0, …, n): // Fixed number of steps
    - g = grad(f, current_conf) // Compute the gradient for a specific configuration
    - current_conf = current_conf – learning_law.val * grad(f, current_conf)
  - learning_law.next()
- return solution

- termination_criteria could either be a fixed number of rounds or checking that $g < \epsilon$.

# Gradient descent

<u>Advantages of the gradient descent :</u>

- Adapted to continuous optimization
- Works well with high dimensional data
- Relatively easy to implement (and efficient)

<u>Disadvantages of the gradient descent :</u>

- The gradient descent method is a **local** heuristic. It is possible to be trapped in local minima.
- Saddle points (nor minimum nor maximum) where the derivative is equal to 0. Big issue seen in parameterized quantum algorithms (see lecture on the QAOA).
- Does not work with function that cannot be derived.

# Simulated annealing

# Simulated Annealing

Origin of simulated annealing:
- Idea of applying the field of **statistical physics** to the field of **combinatorial optimization**.

Idea from Scott Kirkpatrick
(A famous guy who also
worked on Ising problems).
https://www.cs.huji.ac.il/w~kirk/

Analogy to metallurgy :
If a liquid material cools too quickly, then the metal will solidify in a sub-optimal Configuration. If the liquid material cools slowly, the cristal within the material will solidify optimally into a state of minimum energy (called the **ground state**).

=> This is a **minimization problem**, we try to find the configuration of lowest energy

# Simulated Annealing

<u>Simulated annealing:</u>

- At the beginning (high temperature) : nearly random
- At the end (low temperature): looks like a gradient search

- Example with the TSP problem starting from a random initial state :

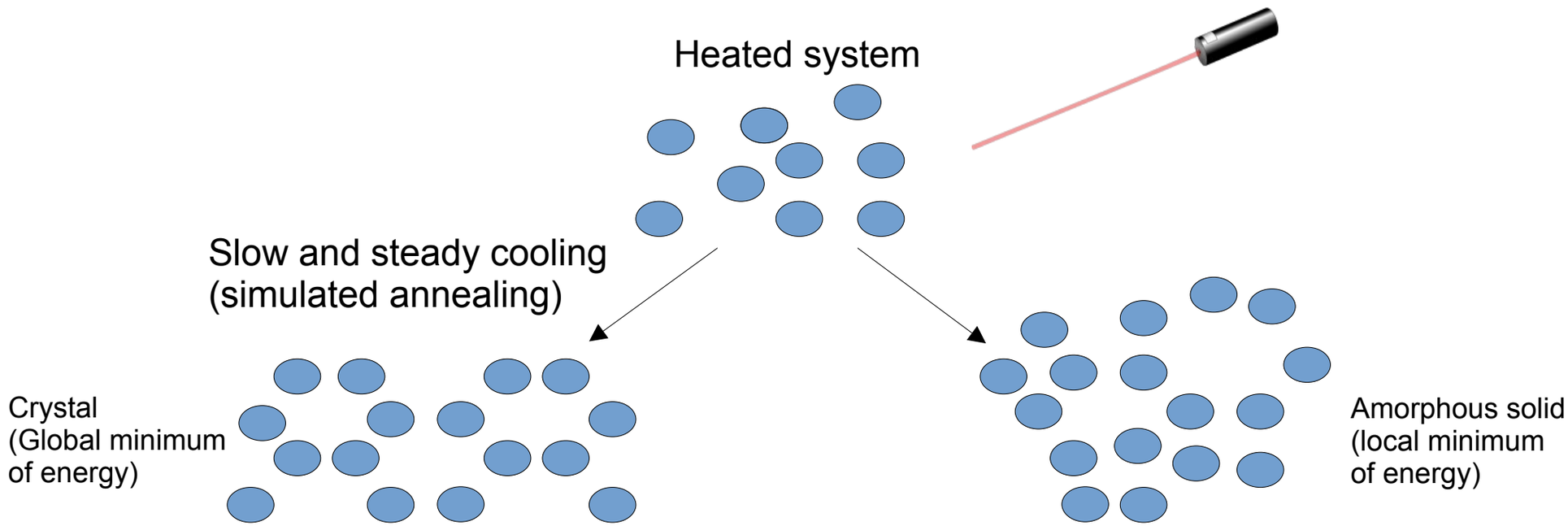
  E : Energy of the solution
  T : Temperature



Source: https://en.wikipedia.org/wiki/Simulated_annealing

# Simulated Annealing

<u>Analogy to metallurgy :</u>



# Simulated Annealing

<u>Simulated annealing:</u>

- At the beginning (high temperature) : nearly random
- At the end (low temperature): looks like a gradient search

- Example with a function :

Source: https://en.wikipedia.org/wiki/Simulated_annealing

# Simulated Annealing

Dilemma:

- We **cannot** compute the energy of all the configurations :
  - Space is often too large (remember, $n!$ for the TSP).
  - The computation time for a single evaluation of energy can be large.

Solution :

- Use the **Metropolis-Hasting algorithm**

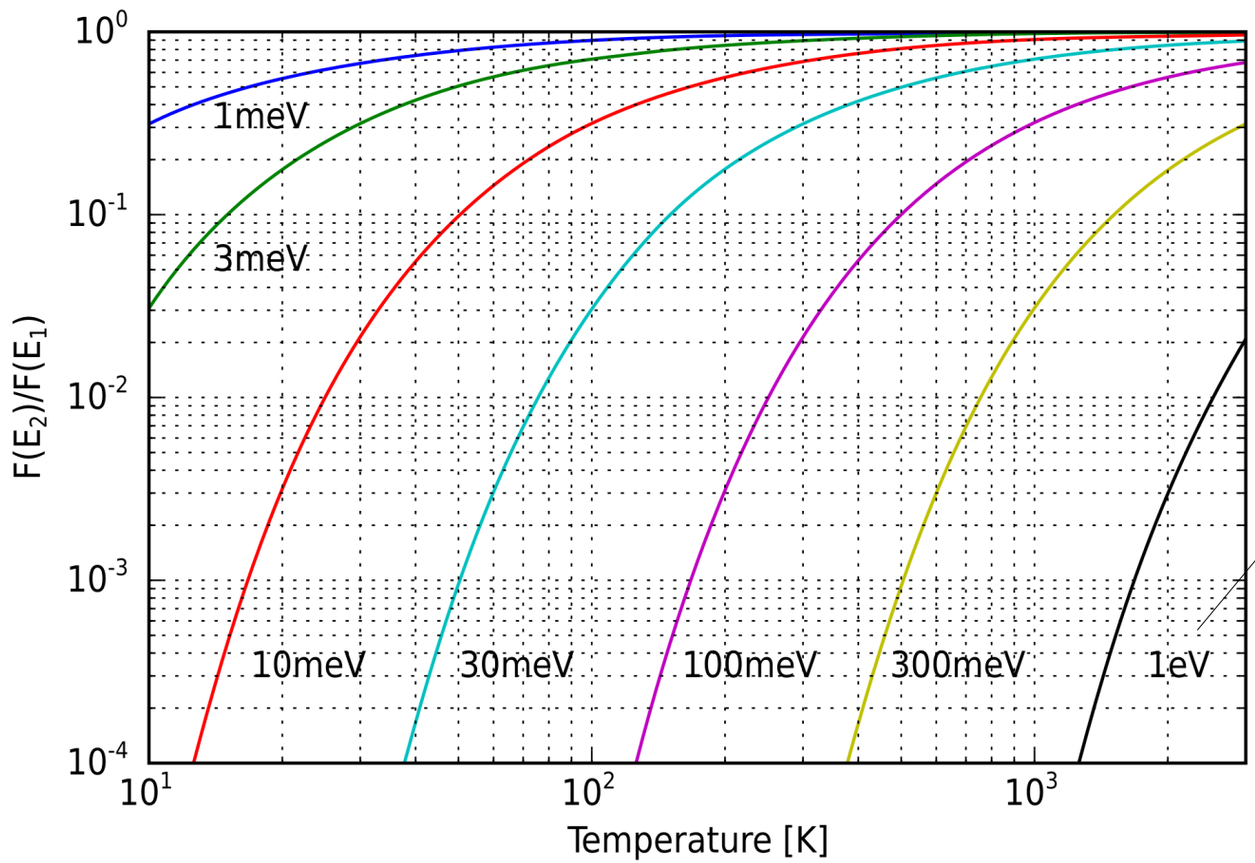
  - **Metropolis** : simulation of the behavior of a set of atoms at thermal equilibrium (no net transfer of energy between the two systems)

  - Probability of a configuration existing at temperature T is given by the acceptance probability function derived from **Boltzmann distribution** (gives the probability that a system will be in a specific state as a function of the state's energy and the temperature of the system)

# Simulated Annealing

Example of Boltzmann curves:

(Defines how much more Likely the state 2 is preferred to the state 1)




Source: https://en.wikipedia.org/wiki/Boltzmann_distribution

High energy difference between state 1 and 2 means we degrade the quality of the solution.

=> low probability at low temperature.

The probability of accepting a state that degrade the quality of the solution decreases exponentially with the temperature

# Simulated Annealing

<u>Acceptance criteria:</u>

- For a given temperature $T$
- Compute the energy $E_{\text{current}}$ of the current configuration current_conf
- Compute the energy $E_{\text{new}}$ of a neighbor configuration new_conf of current_conf

The new configuration is accepted depending on a probability function:

$$P(E_{\text{current}}, E_{\text{new}}, T)$$

When $T \gg 0$, the probability of accepting any new configuration is high, even if it degrades the current configuration quality.

When $T \to 0$, the probability of accepting any new configuration which degrade the current configuration quality is low. Only local improvements are accepted.

In the initial paper of S. Kirkpatrick, the function is defined as:

$$P(E_{\text{current}}, E_{\text{new}}, T) = \exp\left(\frac{-(E_{\text{new}} - E_{\text{current}})}{T}\right)$$

# Simulated Annealing

Algorithm:

Requirements : The algorithm requires the objective function and the configuration space to be defined.

Steps :

- We start in a random initial configuration of the system.
- We then slightly alter the current configuration (current_conf) of the system to create a new configuration (new_conf).
- The energy of each configuration (current_conf) and (new_conf) is compared. If new_conf passes the acceptance criteria (based on probabilities), the system is now considered in current_conf ← new_conf. Otherwise, it stays in current_conf.
- We go back to step 2 and repeat the process many times, decreasing the temperature step by step (several iterations are done for each step to meet thermal equilibrium).
- When the temperature is very low the algorithm stops and the solution is extracted from current_conf.

# Simulated Annealing

Cooling schedule:

The cooling schedule defines how the temperature decreases when every iteration of the annealing step is realized.

Different types of cooling functions:

- Linear cooling:

$$T_{k+1} = T_k - \beta \text{ with } \beta \text{ some constant}$$

- Exponential cooling:

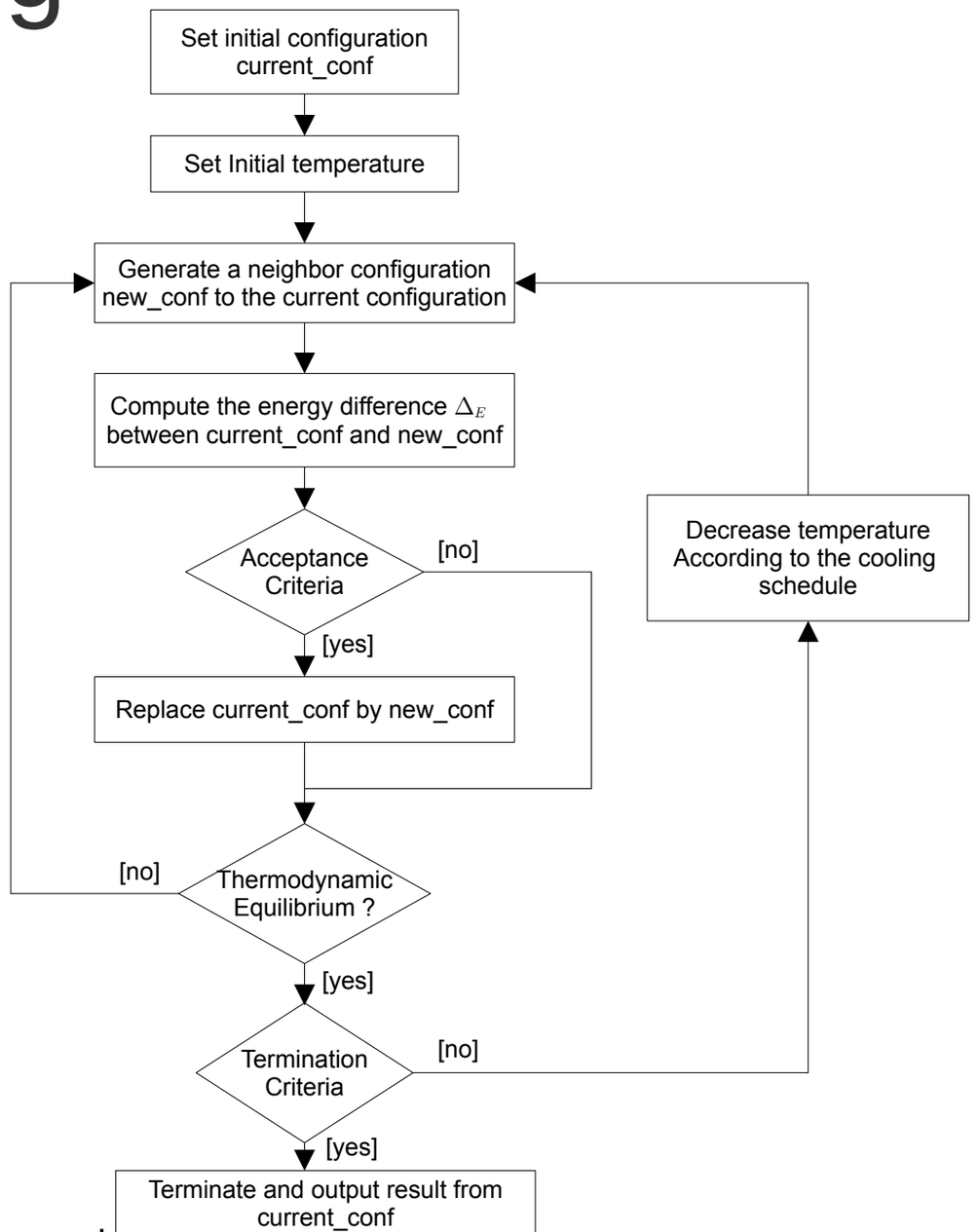
$$T_{k+1} = \beta T_k \text{ with } \beta \text{ some constant} < 1$$

- Logarithmic cooling:

$$T_k = \frac{T_0}{\log(k+1)}$$

# Simulated Annealing

Simulated annealing flow diagram :




**Exercise:** Write the pseudo code for the simulated annealing algorithm

# Simulated Annealing

<u>Simulated annealing pseudo-code :</u>

- current_conf = set_random()
- T = T_START
- While not termination_criteria // when temperature is above a specific threshold
  - While not is_thermodynamic_equilibrium_reached() // fixed number of iterations
    - new_conf = get_random_neighbor(current_conf)
    - $\Delta_E$ = get_energy(new_conf) – get_energy(current_conf)
    - If accept($\Delta_E$)
      - current_conf = new_conf
  - T = decrease_temperature(cooling_schedule, decrease_factor)
- return solution

# Simulated Annealing

<u>Advantages of Simulated Annealing :</u>

- Easy to implement.
- Flexible : work with continuous and discrete optimization.
- Work well on large-size problems.
- The theory (markov chains) shows that the heuristic converges to a global minimum if the temperature does not decrease faster than $C/\log T$ where $C$ is a constant only depending on the critical height (energy barrier which needs to be crossed if you want to reach the global minimum).
  In practice, we use faster schedules because logarithmic decrease is very slow
- Nice iterative algorithm for non convex optimization problems.

# Simulated Annealing

Disadvantages of Simulated Annealing :

- Good parameter values can be tricky to find (temperature, number of temperature steps, and temperature-decreasing law).
- Computing time can be long. The simulated annealing algorithm is inherently iterative and cannot be easily parallelized.
- No guarantee of convergence in practice. The condition for setting the temperature proportional to $C/\log T$ is rarely satisfied in practice due to the long computing time required.

# Branch & Bound

# Branch & Bound

Motivation:
The aim is to solve a problem exactly with a proof that the global optimum has been reached.

However:
- We cannot expect a polynomial-time algorithm to find an exact solution to an NP-complete problem.

- The aim is to **efficiently** build a solution. By efficient, we mean a method that does way better than a brute force method that just list all the possible solutions. Brute force methods are strongly limited by the size of problems.
  But, the global complexity of the method is still exponential.

Main strength of B&B method:
- Leverage constraints of an optimization problem to efficiently explore the solution space.

# Branch & Bound

<u>In general we will consider:</u>

- An objective function:

$$f : \mathcal{X} \to \mathbb{R}$$

- Subject to constraints:

$$\text{such that } x_i \geq y_i$$

<u>Example of problem: The maximum independent set</u>

$$x_v \in \{0, 1\}$$

$$\max C_{MIS} = \sum_{v \in V} x_v$$

$$\text{such that } x_u \neq x_v \text{ if } (u, v) \in E$$

# Branch & Bound

General idea of the algorithm:

- It is possible to limit the number of combinatorial paths by introducing bounds when going through all the possible configurations of the problem.

Definition:

A Branch & Bound algorithm consists of the systematic **enumeration** of candidate solutions where a large number of **useless configurations** are **eliminated** (either because they do not respect the constraints or because the solution cannot reach the best solution already found).
Candidate solutions are eliminated using bounds on the possible values of the cost function.

# Branch & Bound

How the algorithm works concretely:

One strategy is to divide the problem into sub-problems:

- If there exist a method giving lower bounds of cost of any solution part of a subset of all the solutions to the problem.

- If the solution currently found has a better cost than the bound previously computed, it is useless to explore the associated subset of solutions.

The algorithm relies on two procedures that compute an upper and a lower bound on the optimal value of the objective region:

- The upper bound can be defined by evaluating any configuration in the considered region (e.g. random or local search)
- The lower bound can be obtained by relaxation

# Branch & Bound

<u>Example with the Knapsack problem:</u>
Input: $n$ tuples of integers $(w_i, p_i)$ – (weight, benefit)

$$\mathcal{X} = \{0,1\}^n$$

$$\max \sum_i x_i p_i, x_i \in \{0,1\}$$

$$\text{s.t.} \quad \sum_i w_i x_i < W$$

Source: https://en.wikipedia.org/wiki/Knapsack_problem

<u>Instance:</u>

| Object | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| | (2, 5) | (3, 8) | (6, 11) | (4, 9) | (5, 12) | (8, 17) | (4, 8) |

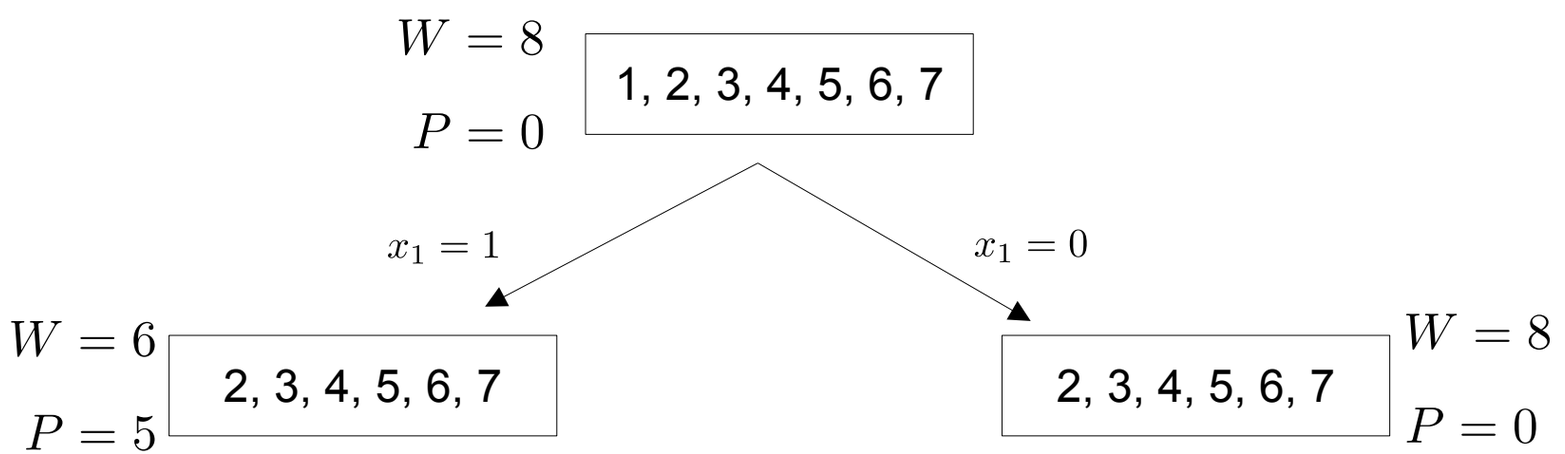
Capacity of the bag: $W = 8$
Total Benefits: $P$

# Branch & Bound

<u>Subproblem division strategy:</u>
We go through all the object sequentially. The branching is done whether the object is selected (1) or not (0).

<u>Let's start with the first object:</u>

Object 1: (2, 5)




If the object 1 is put in the bag, the benefit is $5$ and it remains $W{=}6$

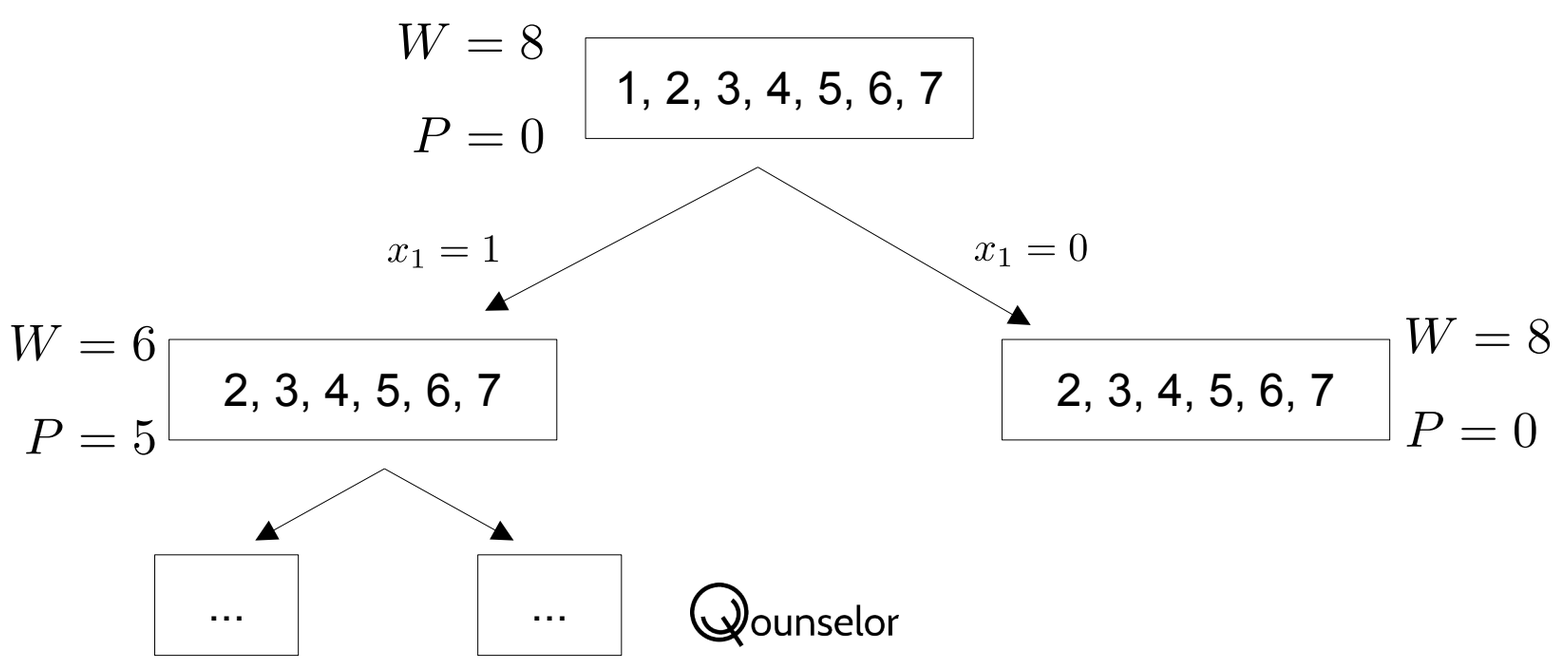
**Exercise:** Can you think of another subproblem division strategy instead of just taking the object sequentially ?

# Branch & Bound

<u>Let's change the division strategy:</u>
We can for example consider the density of each object given by $d_i = \frac{p_i}{w_i}$ and try to fill the bag with the objects having the highest density at first.

| Object | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| | (2, 5) | (3, 8) | (6, 11) | (4, 9) | (5, 12) | (8, 17) | (4, 8) |

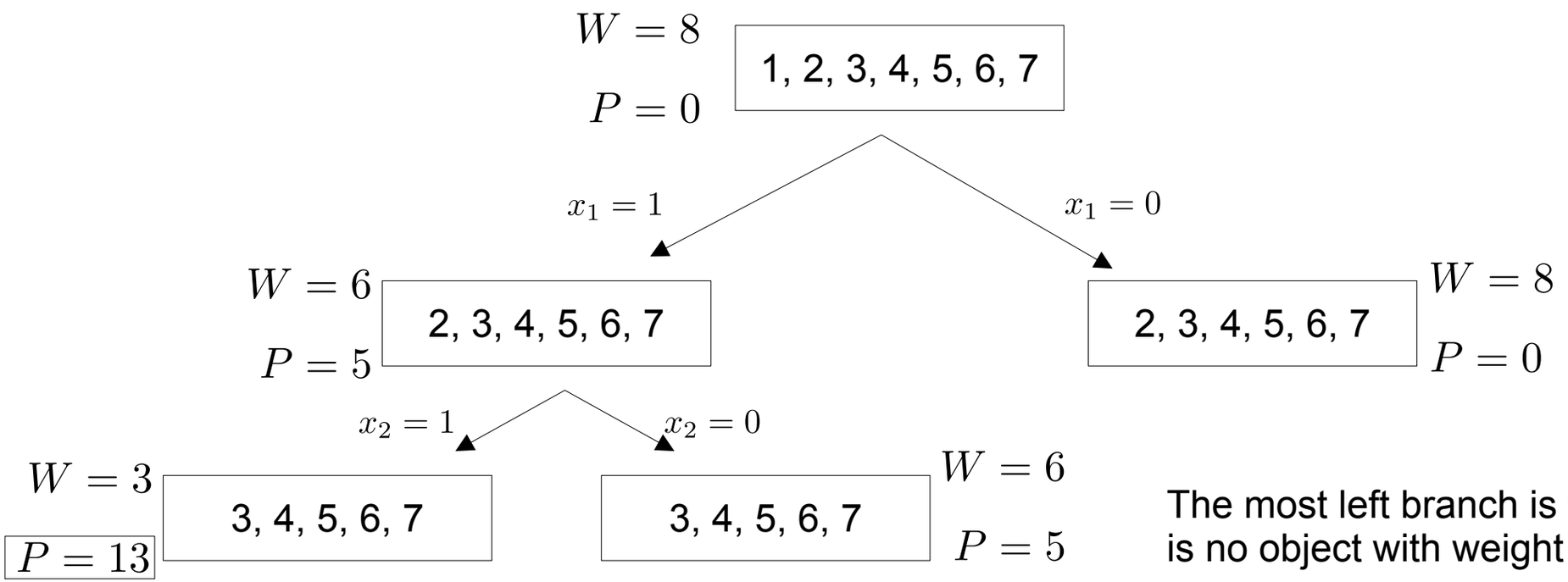
**Exercise:** Compute the density of each object and sub divide the left branch considering this strategy



# Branch & Bound

<u>Objects ordered by density:</u>

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

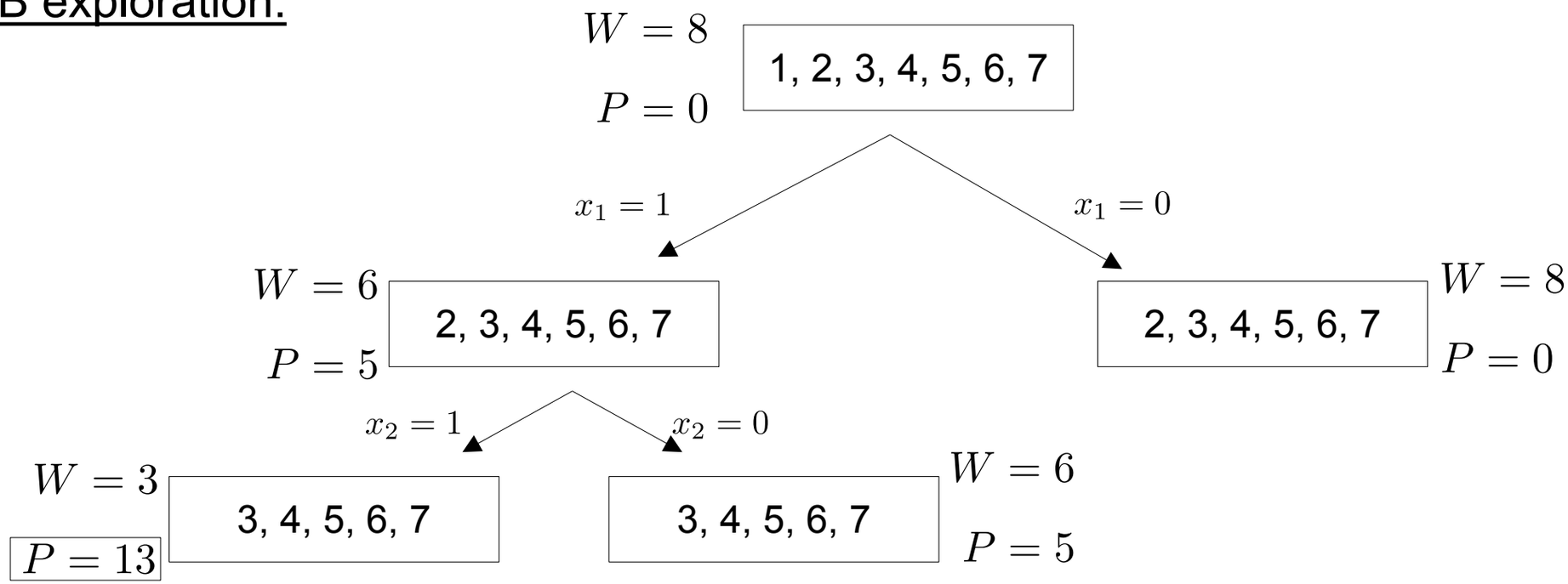



The most left branch is finished as there is no object with weight less than 3

# Branch & Bound

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

B&B exploration:




Computation of the upper bound:

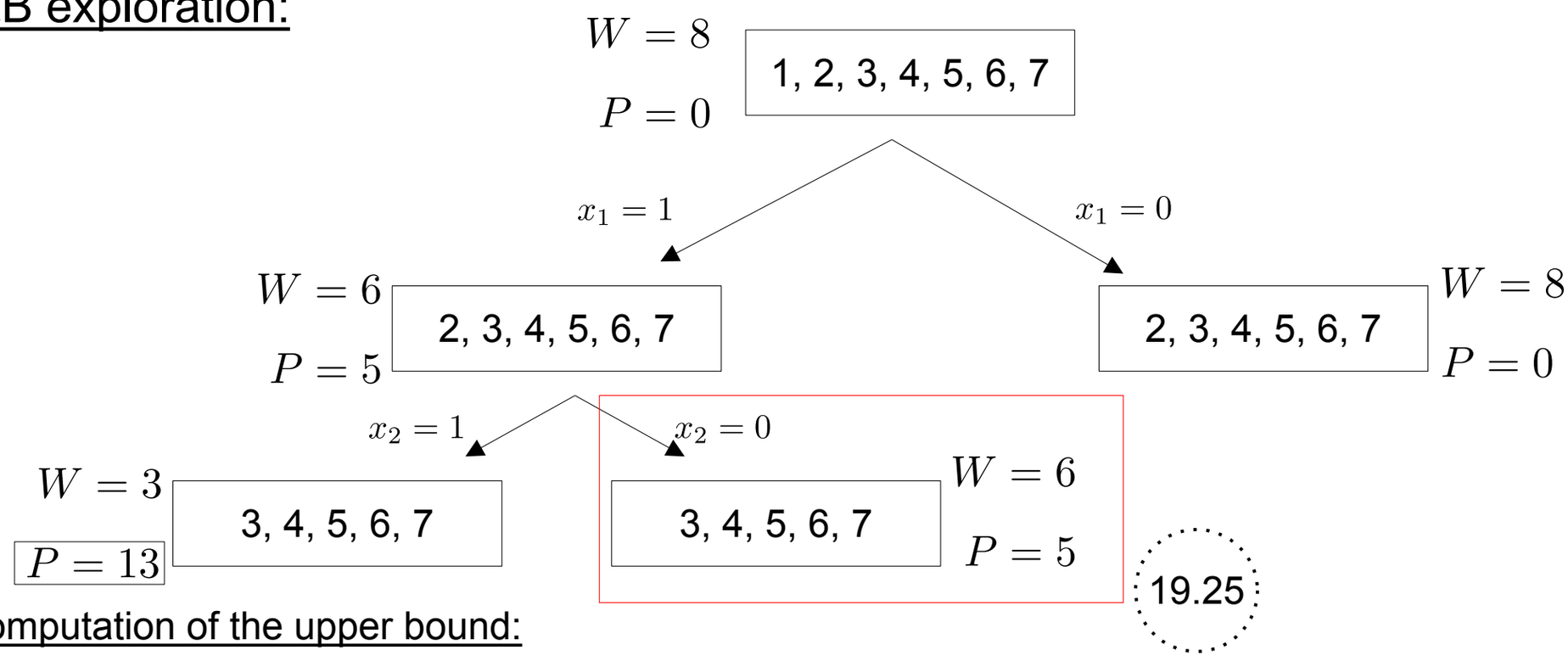
To fill the bag, the best thing we can do is to put the object with the best density that fit in the bag and to complete the rest of the bag with a part of the next object to saturate the bag.

**Exercise:** Compute the upper bound for the branch in the middle

# Branch & Bound

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

B&B exploration:




Computation of the upper bound:

If we put object 5 in the bag, $W{=}1$ and $P{=}12{+}5{=}17$
We then complete the bag with $1/4$ of the object 4: $W{=}0$ and $P{=}17{+}1/4 \times 9 = 19.25$

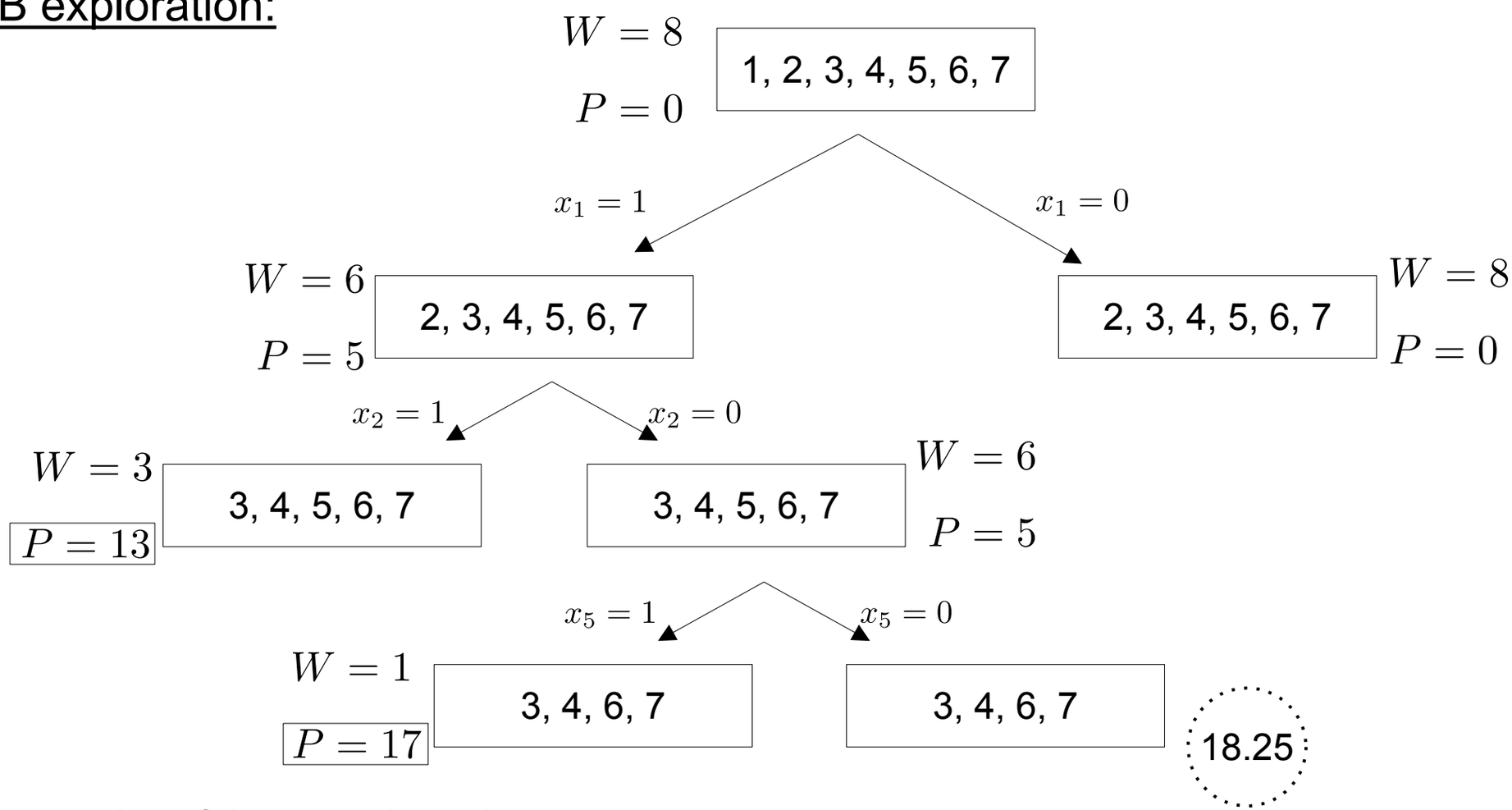
As the lower bound is 13, we must explore the branch, we cannot stop here.

**Exercise:** Continue to explore the branch

# Branch & Bound

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

B&B exploration:



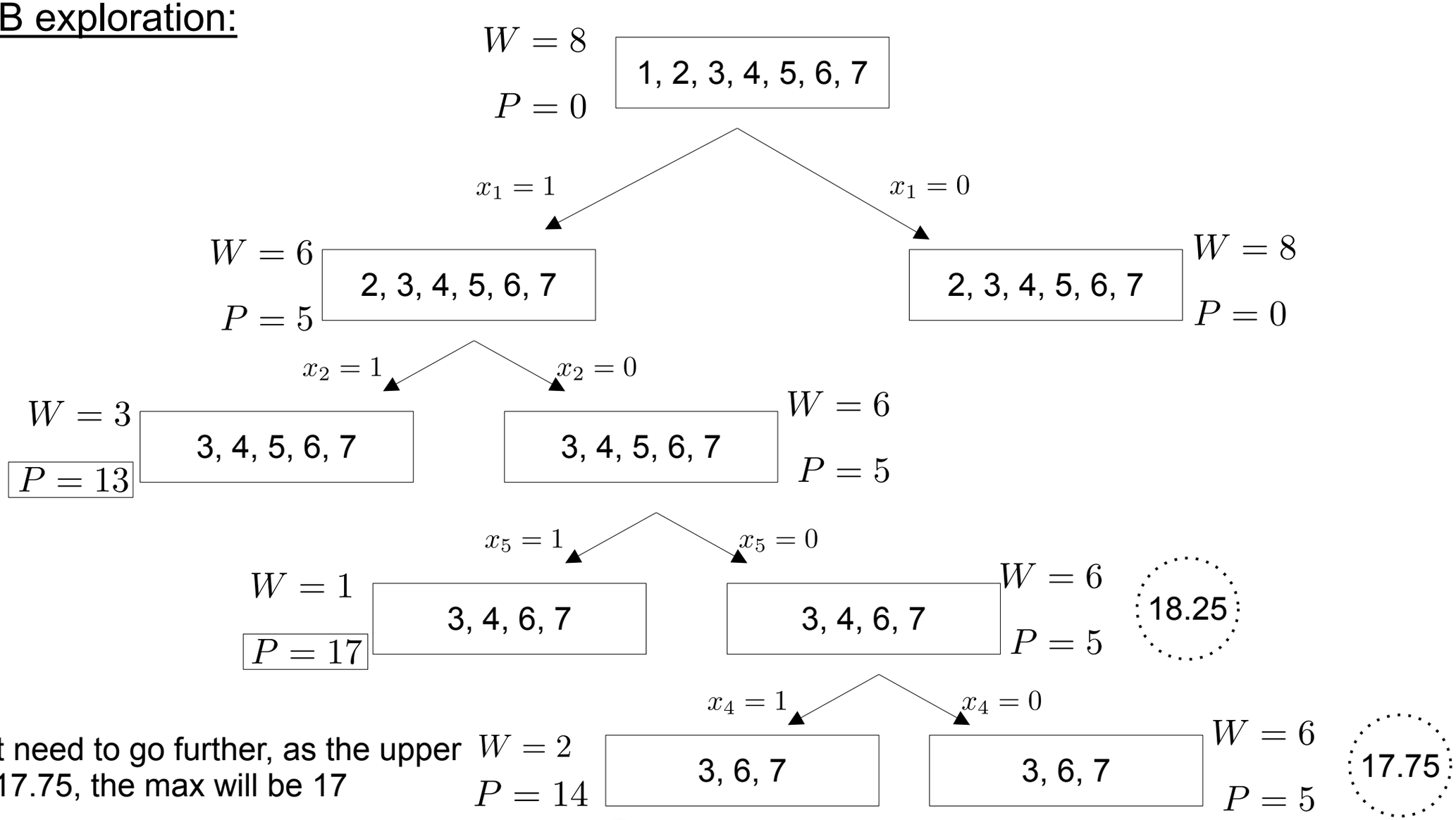

Computation of the upper bound:

$$P = 5 + 9 + 1/4 \times 17 = 18.25$$

# Branch & Bound

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

<u>B&B exploration:</u>

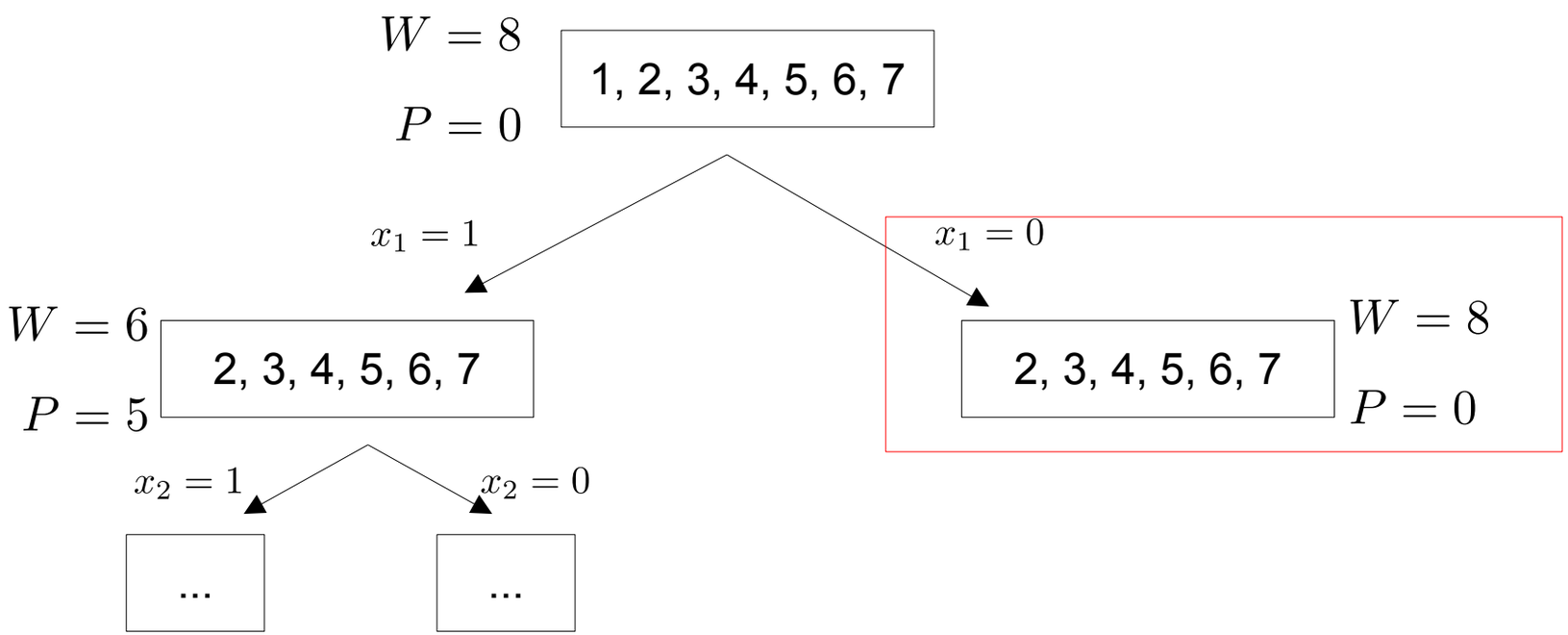



We do not need to go further, as the upper Bound is 17.75, the max will be 17

# Branch & Bound

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

<u>B&B exploration:</u>

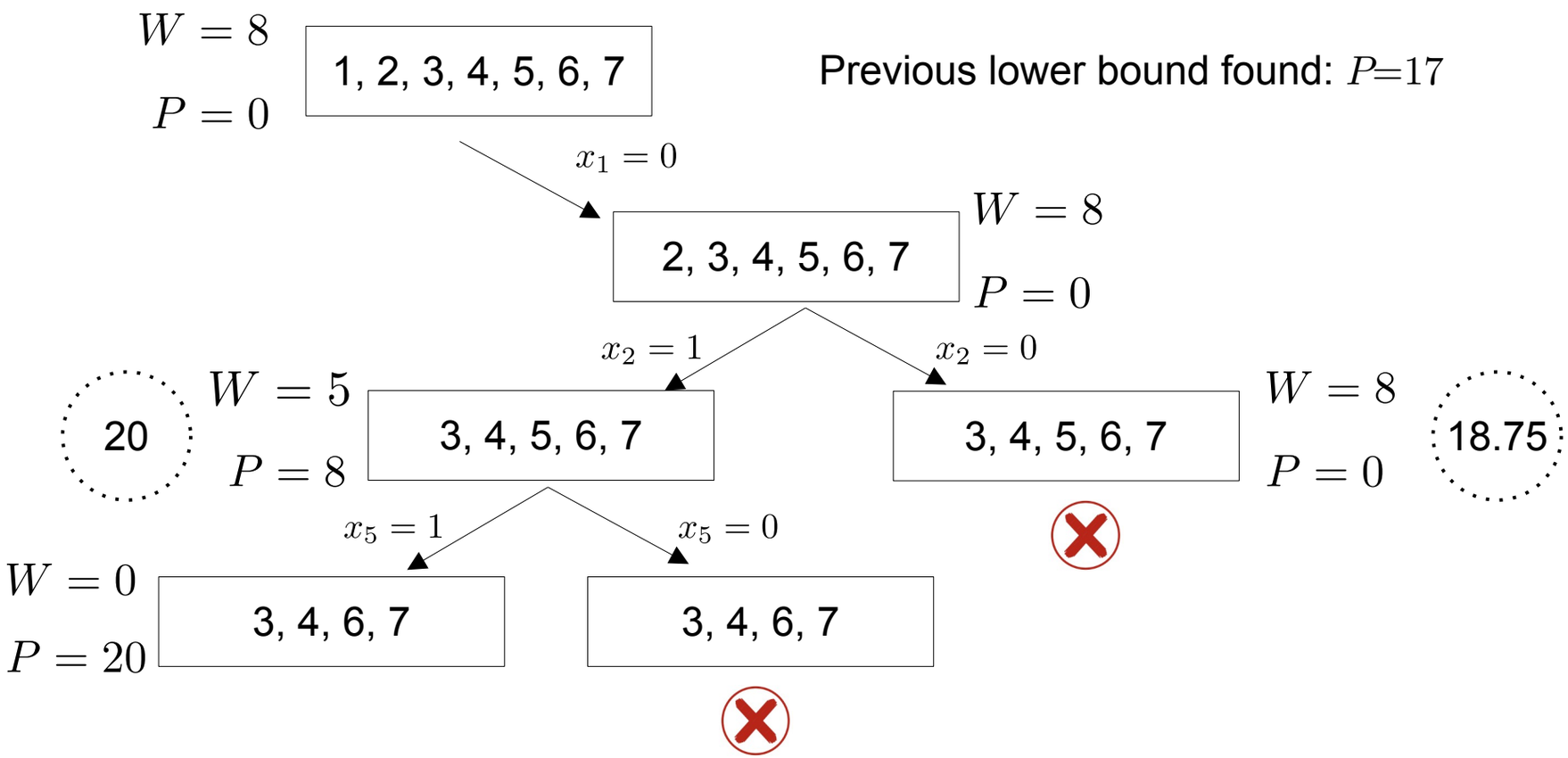



**Exercise:** Explore the second branch

# Branch & Bound

| Object | 2 | 1 | 5 | 4 | 6 | 7 | 3 |
|---|---|---|---|---|---|---|---|
| | (3, 8) | (2, 5) | (5, 12) | (4, 9) | (8, 17) | (4, 8) | (6, 11) |
| Density | 2,67 | 2,5 | 2,40 | 2.25 | 2.12 | 2.00 | 1.83 |

## B&B exploration:




Previous lower bound found: $P$=17

It seems natural to explore the branch with the highest upper bound

=> We have reached the upper bound, we don't even need to explore the rest of the tree

=> The optimal value is $P$=20 and this is **proven** without exploring all the branch in an exhaustive fashion

# Branch & Bound

Advantages of the branch & bound algorithm :

- Optimality: This algorithm guarantees optimality of the solution found
- Efficiency: Can significantly fasten the search by reducing the search space
- Flexibility: Can be adapted to various problem types and domains

Limitations of the branch & bound algorithm :

- Can be complex to implement (especially if specific constraints are involved)
- Heavily relies on parameters choices:
  - Quality of the bounding functions
  - Heuristics used to guide the search
- Not adapted to dynamic and changing environments

# Thank you for your attention !